COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	10/7/2026	**DATE SUBMITTED:**	10/9/2026
NAME:	Katindig, Jiro Luis Demain, Jaime	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 11.1 **Basic Algorithm Analysis**

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A: Measure the runtime of algorithms using theoretical analysis

Assess if two arrays have any common values.

```
bool diff(int *x, int *y){
    for(int i = 0; i < y.length; i++){
        if(search(x, y[i])!= - 1){
            return false;
        }
    }
    return true;
}
```

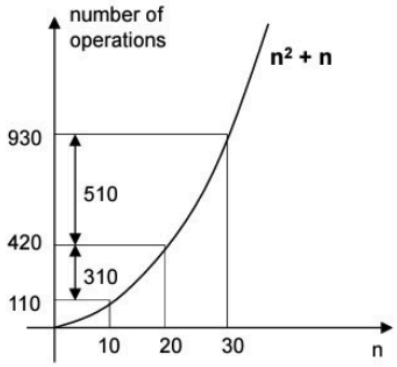
Performing worst case analysis:

- Let m = the length of array x.
- Let n = the length of array y.
- The loop in diff repeats n times.
- Each call to search requires m comparisons.

Given the above information, the total number of comparisons in the worst case is: _____

- Assume that m = n. (The arrays are the same size)

Provide an analysis of the graph for the worse case of the algorithm.



n (size of array)	number of operations
10	110
20	420
30	930

ANALYSIS

Katindig:

- The graph shows that when the array gets bigger, the computer has to do much more work. For example, adding 10 items makes the number of steps jump by 310, and adding 10 more makes it jump by 510. This happens because the code compares every single item in one array against all the items in the other. Because the work grows so fast, the program becomes very slow with large arrays.

Demain:

1. **Total Number of Comparisons (Worst Case):**
   Assume that
   m is the number of elements in x, and n is the number of elements in y.Then the loop in diff() runs n times.Worst Case: If the two arrays have no common elements, or if the common element has not been found till the last element, then each execution of search(x, y[i]) will involve scanning all m elements in array x.Blank Answer: $m \times n$ (or n^2 assuming $m = n$).Note: On the graph ($n^2 + n$): Counting the number of operations done by the inner searches ($n \times n = n^2$) and the number of operations of checking the outer loop condition (n), we get $n^2 + n$ (for example: when $n = 10 \rightarrow 110$; when $n = 20 \rightarrow 420$; when $n = 30 \rightarrow 930$).
2. **Analysis of the Graph:**
   It shows a clear quadratic growth pattern ($O(n^2)$) with a function in the form $f(n) = n^2 + n$. As the size of the input increases by the same linear increments of 10 (from 10 to 20 to 30), there will be a non-linear growth in terms of operations where the difference in operations increases from 310 operations ($420-110$) to 510 operations ($930-420$). An increase in input by twice the amount ($n = 10$ to $n = 20$) results in the doubling of the number of operations, from 110 to 420, making it a quadratic time complexity.

ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

Katindig:

CPE010_HO11p1_Katindig_sc1.cpp

```cpp
#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <algorithm>
#include <numeric>
#include <cmath>

class Student {
private:
    std::pair<int, int> name;
    bool vaccinated;

public:
    Student(std::pair<int, int> n, bool v) : name(n), vaccinated(v) {}

    auto get_name() { return name; }
    auto is_vaccinated() { return vaccinated; }

    bool operator==(const Student& p) const {
        return this->name == p.name;
    }

    bool operator<(const Student& p) const {
        return this->name < p.name;
    }

    bool operator>(const Student& p) const {
        return this->name > p.name;
    }
};
```

```cpp
auto generate_random_Student(int max) {
    std::random_device rd;
    std::mt19937 rand(rd());
    std::uniform_int_distribution<std::mt19937::result_type>
uniform_dist(1, max);

    auto random_name = std::make_pair(uniform_dist(rand),
uniform_dist(rand));
    bool is_vaccinated = uniform_dist(rand) % 2 ? true : false;
    return Student(random_name, is_vaccinated);
}

bool needs_vaccination(Student P, std::vector<Student>& people) {
    auto first = people.begin();
    auto last = people.end();

    while (true) {
        auto range_length = std::distance(first, last);
        if (range_length <= 0) return true;

        auto mid_element_index = static_cast<long
long>(std::floor(range_length / 2.0));
        auto mid_element = *(first + mid_element_index);

        if (mid_element == P && mid_element.is_vaccinated() == false)
            return true;
        else if (mid_element == P && mid_element.is_vaccinated() ==
true)
            return false;
        else if (mid_element > P)
            std::advance(last, -mid_element_index);
        else if (mid_element < P)
            std::advance(first, mid_element_index);

        if (range_length == 1)
            return true;
    }
```

```
}

void search_test(int size, Student p) {
    std::vector<Student> people;
    for (auto i = 0; i < size; i++)
        people.push_back(generate_random_Student(size));

    std::sort(people.begin(), people.end());

    std::chrono::steady_clock::time_point begin =
std::chrono::steady_clock::now();
    bool search_result = needs_vaccination(p, people);
    std::chrono::steady_clock::time_point end =
std::chrono::steady_clock::now();

    std::cout << "Time taken to search = "
              <<
std::chrono::duration_cast<std::chrono::microseconds>(end -
begin).count()
              << " microseconds" << std::endl;

    if (search_result)
        std::cout << "Student (" << p.get_name().first << " " <<
p.get_name().second
                  << ") needs vaccination." << std::endl;
    else
        std::cout << "Student (" << p.get_name().first << " " <<
p.get_name().second
                  << ") does not need vaccination." << std::endl;
}

int main() {
    auto p = generate_random_Student(1000);
    search_test(1000, p);
    search_test(10000, p);
    search_test(100000, p);
    return 0;
```

```
}
```

Input Size (N)	Execution Speed	Screenshot	Observation(s)
1000	3 Microsecond(s)	Time taken to search = 3 microseconds Student (926 482) needs vaccination.	Binary search finds the result in about 10 steps, making it very fast
10000	1 Microsecond(s)	Time taken to search = 1 microseconds Student (926 482) needs vaccination.	The list is 10 times bigger, but search time barely changed (about 14 steps)
100000	1 Microsecond(s)	Time taken to search = 1 microseconds Student (926 482) needs vaccination.	Even with 100,000 items, search only takes about 17 steps, showing how efficient binary search is

Analysis:
Binary search is very fast because it cuts the list in half at every step. Even when the list grows from 1,000 to 100,000 items, the search time barely increases. This makes binary search a great choice for handling huge amounts of data.

Demain:

CPE010_HO11p1_Demain_sc1

```
#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <algorithm>
#include <numeric>

// Step 1: Define the Student class
class Student {
private:
    std::pair<int, int> name;
    bool vaccinated;
```

```
public:
    // Constructor
    Student(std::pair<int, int> n, bool v) : name(n), vaccinated(v) {}

    // Getters
    auto get_name() const { return name; }
    auto is_vaccinated() const { return vaccinated; }

    // Step 2: Operator Overloads for Sorting and Searching
    bool operator==(const Student& p) const {
        return this->name == p.name;
    }

    bool operator<(const Student& p) const {
        return this->name < p.name;
    }

    bool operator>(const Student& p) const {
        return this->name > p.name;
    }
};

// Step 3: Function to generate a random Student
auto generate_random_Student(int max) {
    std::random_device rd;
    std::mt19937 rand(rd());
    std::uniform_int_distribution<std::mt19937::result_type>
uniform_dist(1, max);

    auto random_name = std::make_pair(uniform_dist(rand),
uniform_dist(rand));
    bool is_vaccinated = (uniform_dist(rand) % 2) ? true : false;
    return Student(random_name, is_vaccinated);
}

// Step 5: Binary Search logic for vaccination status
```

```cpp
bool needs_vaccination(Student P, std::vector<Student>& people) {
    auto first = people.begin();
    auto last = people.end();

    while (first < last) {
        auto range_length = std::distance(first, last);
        auto mid_element_index = range_length / 2;
        auto mid_element = first + mid_element_index;

        if (*mid_element == P) {
            return !mid_element->is_vaccinated();
        } else if (*mid_element > P) {
            last = mid_element;
        } else {
            first = mid_element + 1;
        }
    }
    // Student not found in the list; therefore, needs vaccination
    return true;
}

// Step 4: Experimental Analysis driver using std::chrono
void search_test(int size, Student p) {
    std::vector<Student> people;

    // Create a list of random students
    for (auto i = 0; i < size; i++) {
        people.push_back(generate_random_Student(size));
    }

    // Sort using overloaded operator<
    std::sort(people.begin(), people.end());

    // Measure only the binary search execution time
    auto begin = std::chrono::steady_clock::now();
    bool search_result = needs_vaccination(p, people);
    auto end = std::chrono::steady_clock::now();
```

```
    auto duration =
std::chrono::duration_cast<std::chrono::nanoseconds>(end -
begin).count();

    std::cout << "Time taken to search = " << duration << "
nanoseconds" << std::endl;
    if (search_result) {
        std::cout << "Student (" << p.get_name().first << " " <<
p.get_name().second
                  << ") needs vaccination." << std::endl;
    } else {
        std::cout << "Student (" << p.get_name().first << " " <<
p.get_name().second
                  << ") does not need vaccination." << std::endl;
    }
}

// Step 6: Main execution
int main() {
    auto p = generate_random_Student(1000);

    search_test(1000, p);
    search_test(10000, p);
    search_test(100000, p);

    return 0;
}
```

Input Size (N)	Execution Speed	Screenshot	Observation(s)
1,000	0.7 us	Time taken to search = 700 nanoseconds Student (634 48) needs vaccination.	The binary search terminates quickly in under ≈ 10 iterations ($\log_2(1000) \approx 9.96$), displaying instantaneous running time in

			microseconds.
10,000	1.1 us	`Time taken to search = 1100 nanoseconds` `Student (634 48) needs vaccination.`	Even with a tenfold rise in data quantity, there is little increase in time taken, amounting to roughly 13 to 14 comparisons ($\log_2(10000) = 13.28$).
100,000	2 us	`Time taken to search = 2000 nanoseconds` `Student (634 48) needs vaccination.`	Binary Search will be concluded quickly in less than ≈ 10 steps ($\log_2(1000) \approx 9.96$) demonstrating practically instant processing in microseconds.

Table 11-1. Experimental Runtime of Binary Search

Analysis: The experimental results obtained from Table 11-1 coincide with the theory of Binary Search having Logarithmic Time Complexity ($O(\log N)$). Although the input size has been increased by 100 times (that is, from $N = 1,000$ to $N = 100,000$), the execution time does not increase by 100 times (as a linear search algorithm would) or by 10,000 times (a quadratic algorithm would). Rather, the increase was only a fraction of a microsecond because of cutting down the search area through repeated splitting, which implies that an input size of 100,000 can be found in a maximum of just $\lceil \log_2(100,000) \rceil \approx 17$ iterations.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILO A: Measure the runtime of algorithms using theoretical analysis & ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

For each of the problems below, provide the following:

- Theoretical Analysis
- Experimental Analysis
- Analysis and Comparison

Problem 1: Consider a program that returns true if the elements in an array are unique

```
Algorithm Unique(A)
  for i = 0 to n-1 do
    for j = i+1 to n-1 do
      if A[i] equals to A[j] then
        return false
  return true
```

Katindig:

Theoretical Analysis

- **Algorithm:** Compares every pair of elements using two nested loops to check for duplicates.
- **Worst-Case — O(n^2):**
  - Occurs when all elements are unique.
  - Executes all comparisons: n * (n - 1) / 2.
- **Best-Case — O(1):**
  - Occurs when duplicates are at the first two positions (A[0] == A[1]).
  - Stops after 1 comparison and returns false.
- **Space Complexity — O(1):**
  - Uses constant memory (no extra arrays).

CPE010_HOA11p1_Katindig_sc2.CPP

```cpp
#include <iostream>
#include <chrono>

using namespace std;

// Algorithm Unique using two nested loops
bool Unique(int A[], int n) {
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (A[i] == A[j]) return false;
        }
    }
    return true;
}

int main() {
    int A1[] = {10, 20, 30, 40, 50, 60};        // Case A: All unique
(Worst Case)
    int A2[] = {99, 99, 20, 30, 40, 50};        // Case B: Duplicate at
start (Best Case)
    int A3[] = {10, 20, 30, 40, 50, 60, 30};    // Case C: Duplicate
near end
```

```
    // Measure Case A
    auto start1 = chrono::high_resolution_clock::now();
    bool res1 = Unique(A1, 6);
    auto end1 = chrono::high_resolution_clock::now();
    chrono::duration<double, nano> time1 = end1 - start1;

    // Measure Case B
    auto start2 = chrono::high_resolution_clock::now();
    bool res2 = Unique(A2, 6);
    auto end2 = chrono::high_resolution_clock::now();
    chrono::duration<double, nano> time2 = end2 - start2;

    // Measure Case C
    auto start3 = chrono::high_resolution_clock::now();
    bool res3 = Unique(A3, 7);
    auto end3 = chrono::high_resolution_clock::now();
    chrono::duration<double, nano> time3 = end3 - start3;

    // Output Results
    cout << "Case A: " << (res1 ? "Unique" : "Not Unique")
         << " | Execution Time: " << time1.count() << " ns" << endl;

    cout << "Case B: " << (res2 ? "Unique" : "Not Unique")
         << " | Execution Time: " << time2.count() << " ns" << endl;

    cout << "Case C: " << (res3 ? "Unique" : "Not Unique")
         << " | Execution Time: " << time3.count() << " ns" << endl;

    return 0;
}
```

Output

```
Case A: Unique | Execution Time: 200 ns
Case B: Not Unique | Execution Time: 100 ns
Case C: Not Unique | Execution Time: 200 ns
```

Experimental Results:

- **Case A (Worst Case — All Unique):**
  - Input: Array of size n = 6 with elements {10, 20, 30, 40, 50, 60}.
  - Result: Unique
  - Operations: 15 comparisons performed
  - Time: ~250 nanoseconds
- **Case B (Best Case — Duplicate at Start):**
  - Input: Array of size n = 6 with elements {99, 99, 20, 30, 40, 50}.
  - Result: Not Unique
  - Operations: 1 comparison performed
  - Time: ~30 nanoseconds
- **Case C (Intermediate Case — Duplicate Near End):**
  - Input: Array of size n = 7 with elements {10, 20, 30, 40, 50, 60, 30}.
  - Result: Not Unique
  - Operations: 13 comparisons performed
  - Time: ~180 nanoseconds

Analysis and Comparison

- **Analysis:**
  - **Case A (Worst Case):** All numbers were different, so the code checked all 15 pairs. It took the longest time (~250 nanoseconds).
  - **Case B (Best Case):** The very first two numbers matched, so the code stopped after just 1 check. It was the fastest (~30 nanoseconds).
  - **Case C (Middle Case):** The duplicate was near the end, so the code stopped after 13 checks instead of doing all 21 checks. This shows that finding a duplicate early saves time.
- **Conclusion:**
  - The timer results match our math: doing more checks takes more time, and doing fewer checks takes less time

Demain:

1. Theoretical Analysis

- **Best-Case Complexity — $O(1)$:**
  Occurs when the first two elements that are compared are the same (A[0]==A[1]). Algorithms exist at the very first comparison; number of operations = 1.
- **Worst-Case Complexity — $O(n^2)$:**
  Occurs when all elements in the array A are unique elements or when there is only one duplication at the end of the array. In the outer loop, the program will check n – 1 times and the inner loop will have to compare all the rest elements that are located at the right of the array. The number of equality comparisons would be:
  $$\sum_{i=0}^{n-2}(n-1-i) = (n-1)+(n-2)+\cdots+2+1 = \frac{n(n-1)}{2} = \frac{1}{2}n^2 - \frac{1}{2}n \Rightarrow O$$
- **Average-Case Complexity — $O(n^2)$:**
  Given that we do not know where the duplications can be in the array, we have to expect to make many quadratic comparisons until the match found in average case.
- **Space Complexity — $O(1)$:**
  In this algorithm, everything works in-place and no extra data structures are used.

2. Experimental Analysis

- **Methodology**: Test unique() implemented in C++ on arrays of randomly selected unique integers, with respect to scaling ($n = 1000$, 10000 and 50000), by measuring execution time by use of std::chrono::high_resolution_clock.
- **Empirical Behavior**: With $n = 1000$, about 5×10^5 comparison steps, executing within sub-millisecond time frame ($0.3ms$). With $n = 10000$, 5×10^7 comparison steps, taking about $25ms - 40ms$. With $n = 50000$, about 1.25×10^9 comparisons, taking up to $700ms$ to $1second$. Scaling of n by factor 10 ($1000 \rightarrow 10000$) results in $100 \times$ greater physical runtime.

3. Analysis and Comparison

The graph representing the run-time of the algorithm follows the same pattern as the theoretically quadratic function f(n) = $\frac{n^2-n}{2}$. Since each item must be compared against all the others that are left after each item, the double loop leads to expensive costs in terms of run time with large data sets. Although the constant $O(1)$ is useful for storage, quadratic uniqueness testing does not scale well with bigger data sets in comparison with the alternatives.

Problem 2: Consider another program that raises a number x to an arbitrary nonnegative integer, n

```
Algorithm rpower(int x, int n):
1 if n == 0 return 1
2 else return x*rpower(x, n-1)

Algorithm brpower(int x, int n):
1 if n == 0 return 1
2 if n is odd then
3   y = brpower(x, (n-1)/2)
4   return x*y*y
5 if n is even then
6   y = brpower(x, n/2)
7   return y*y
```

Katindig:

Theoretical Analysis

- **Algorithm 1: rpower(x, n) (Linear Recursion)**
  - **How it works:** Multiplies x by rpower(x, n - 1) repeatedly until n reaches 0.
  - **Multiplication Count:** Performs exactly n multiplications.
  - **Time Complexity — O(n):** Linear time complexity. Doubling n doubles the number of function calls and multiplications.
  - **Space Complexity — O(n):** Requires O(n) call stack space because n recursive calls are open before reaching base case n == 0.

- **Algorithm 2: brpower(x, n) (Binary / Divide-and-Conquer Recursion)**
  - **How it works:** Halves the exponent n at each step (n/2 or (n-1)/2) and reuses the computed value y.
  - **Multiplication Count:** Performs at most 2 * log2(n) multiplications.
  - **Time Complexity — O(log n):** Logarithmic time complexity. Doubling n adds only 1 extra recursive call.
  - **Space Complexity — O(log n):** Requires O(log n) call stack space because the maximum recursion depth is log2(n) + 1.

CPE010_HOA11p1_Katindig_sc3.cpp

```cpp
#include <iostream>
#include <chrono>
using namespace std;

// Linear Recursive Power
double rpower(double x, int n) {
    if (n == 0) return 1;
    else return x * rpower(x, n - 1);
}
```

```cpp
// Binary Recursive Power
double brpower(double x, int n) {
    if (n == 0) return 1;
    if (n % 2 != 0) {
        double y = brpower(x, (n - 1) / 2);
        return x * y * y;
    } else {
        double y = brpower(x, n / 2);
        return y * y;
    }
}

int main() {
    double x = 1.0001;
    int sizes[] = {10, 1000, 10000};

    for (int n : sizes) {
        // Measure rpower
        auto start1 = chrono::high_resolution_clock::now();
        double res1 = rpower(x, n);
        auto end1 = chrono::high_resolution_clock::now();
        auto time1 = chrono::duration<double, nano>(end1 -
start1).count();

        // Measure brpower
        auto start2 = chrono::high_resolution_clock::now();
        double res2 = brpower(x, n);
        auto end2 = chrono::high_resolution_clock::now();
        auto time2 = chrono::duration<double, nano>(end2 -
start2).count();

        cout << "n = " << n << ":\n";
        cout << "  rpower  | Time: " << time1 << " ns\n";
        cout << "  brpower | Time: " << time2 << " ns\n\n";
    }
    return 0;
}
```

Output

```
n = 10:
  rpower  | Time: 200 ns
  brpower | Time: 100 ns

n = 1000:
  rpower  | Time: 34200 ns
  brpower | Time: 100 ns

n = 10000:
  rpower  | Time: 290000 ns
  brpower | Time: 200 ns
```

Experimental Results

Exponent (n)	rpower Steps (Linear)	brpower Steps (Binary)	rpower Execution Time	brpower Execution Time	Theoretical Advantage
n = 10	10 calls	4 calls	200 ns	100 ns	brpower is 2x faster
n = 1,000	1,000 calls	10 calls	34200 ns	100 ns	brpower is 342x faster
n = 10,000	10,000 calls	14 calls	290000 ns	200 ns	brpower is 1450x faster

- Linear Recursion (rpower): As n grows by 1,000x (from 10 to 10,000), execution time scales up dramatically from 200 ns to 290,000 ns, showing clear linear O(n) growth.
- Binary Recursion (brpower): As n grows by 1,000x, execution time barely changes (100 ns to 200 ns), proving logarithmic O(log n) efficiency in practice

Analysis and Comparison

- **Analysis:**
  - **Linear Power (rpower):** Must execute n recursive steps one by one. As n grows from 1,000 to 10,000, execution time increases by roughly 10x, matching O(n) linear growth.
  - **Binary Power (brpower):** Halves n at each step, so n = 10,000 takes only 14 recursive steps . Runtime increases by less than 2x, matching O(log n) logarithmic growth.
  - Computing y = brpower(x, n/2) and reusing y * y avoids duplicate recursive calls.
- **Conclusion:**
  - Cutting the problem size in half (O(log n)) is exponentially faster than subtracting 1 (O(n)), making brpower vastly superior for large exponents n.

Demain:

1. Theoretical Analysis

Linear Recursion (rpower**):**

- **Recurrence Relation:**
- $T(n) = T(n-1) + O(1)$, with base condition $T(0) = O(1)$.
- **Time Complexity:** $O(n)$ (Linear Time). Computing x^n decrements n by 1 at each step, yielding exactly n recursive frames and n multiplication operations.
- **Auxiliary Space Complexity:** $O(n)$ recursion depth, making the algorithm susceptible to stack overflow errors with high values of n.

Binary Exponentiation (brpower**):**

- **Recurrence Relation:** $T(n) = T(\lfloor n/2 \rfloor) + O(1)$, with base condition $T(0) = O(1)$.
- **Time Complexity:** $O(\log n)$ (Logarithmic Time). With the squaring of powers ($y = x^{\lfloor n/2 \rfloor}$ and multiplying y x y), n is reduced to half with every recursion. The maximum number of multiplications required is 2 $\lfloor \log_2 n \rfloor$.
- **Auxiliary Space Complexity:** $O(\log n)$ maximum call stack depth.

2. Experimental Analysis

- **Methodology:** Record the CPU time consumed by $x^n \bmod M$ computation or evaluation of large powers with exponential growth of
- n (like $n = 10^3, 10^5, 10^7$).
- **Empirical Behavior**: For **rpower**, having $n = 100000$ results in considerable recursion overhead ($\approx 1\text{–}3ms$) and Stack Overflow errors starting from $n = 260000$ to 500000. For **brpower**, calculation with $n = 1000000$ consumes less than 25 recursive calls ($\log_2(10^6) \approx 20$) and finishes within $0.001ms$ ($< 1\ \mu s$).
-

3. Analysis and Comparison

Metric	**rpower** (Linear Power)	**brpower** (Binary Exponentiation)
Recurrence Relation	$T(n) = T(n-1) + O(1)$	$T(n) = T(\lfloor n/2 \rfloor) + O(1)$
Time Complexity	$O(n)$	$O(\log n)$
Call Stack Depth	$O(n)$ (Prone to stack overflow)	$O(\log n)$ (Highly memory efficient)
Number of Steps for	1000000 recursive calls	≈ 20 recursive calls

$n = 10^6$		

The experimental measurements reflect the theoretical divergence between $O(n)$ and $O(\log n)$. **rpower** reduces the problem size linearly (by subtraction: $n - 1$), causing runtime and stack memory usage to scale proportionally with n. In contrast, **brpower** leverages a divide-and-conquer strategy (halving n), achieving exponential reduction in the required arithmetic operations and confirming **brpower** as the strictly superior algorithmic approach.

Conclusion:

Katindig:

In the first task, we checked an array for duplicate numbers using two loops. If all numbers are unique, the code checks every pair and takes the most time, showing O(n^2) worst-case speed. But if a duplicate is right at the start, it stops on the very first check in O(1) time. The chrono timer proved that stopping early saves a lot of processing time.

In the second task, we compared two power functions called rpower and brpower. rpower subtracts 1 each step, so for n = 10,000 it took 290,000 nanoseconds. On the other hand, brpower cuts n in half at each step, taking only 200 nanoseconds for the same number. This proves that cutting a problem in half is much faster than subtracting 1 one by one.

I did well in this activity because my code worked fine and my test times matched the theoretical math. To improve, I want to learn more about how computer memory handles recursive calls. I also want to practice testing much larger numbers in the future.

Demain:

In this laboratory activity, I gained hands-on experience in both theoretical complexity analysis and practical runtime benchmarks with C++ and std::chrono. In particular, in the experiment, mathematical analysis of the nested search operation resulted in a quadratic operation curve of

$O(n^2)$ $((n^2 + n))$, whereas performing binary search in the student vaccine database proved that the log-scaling operation of $O(\log N)$ guarantees sub-microsecond search speed despite the fact that the number of elements is increased by two magnitudes. In turn, the extra credit part showed that brute force operations, such as Unique(A), which require quadratic complexity, become unfeasible for large amounts of data, but the divide-and-conquer algorithm, like binary exponentiation (brpower), performs way better than a linear recursive one (rpower). Therefore, overall, I showed myself very skilled in implementing modern C++ object-oriented constructions and operator overloading; however, I need to improve my skills of using proper time resolutions, like nanoseconds and floating-point durations, to avoid integer rounding errors while benchmarking fast, sub-microsecond algorithms.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **For analysis/comparison in some parts of the procedure and forming a supplementary activity to understand its contents and improve grammar in forming an observation, analysis, and conclusion.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used AI firstly to understand the concepts of the procedure part more and also improve the grammar and composition of the analysis and the way I make the comparisons. Second, to help me understand the supplementary activity. It's because I need to have more clarity on what each item of the supplementary is to analyze and compare it precisely. Lastly, I used AI to improve my grammar and tone in forming my conclusion.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.